

รายงานโครงงาน

HONEY RESTAURANT

แอปสั่งอาหารในร้าน

วิชา 01418342
การพัฒนาแอปพลิเคชันบนอุปกรณ์เคลื่อนที่
กลุ่มที่ 4 เคบับKAI

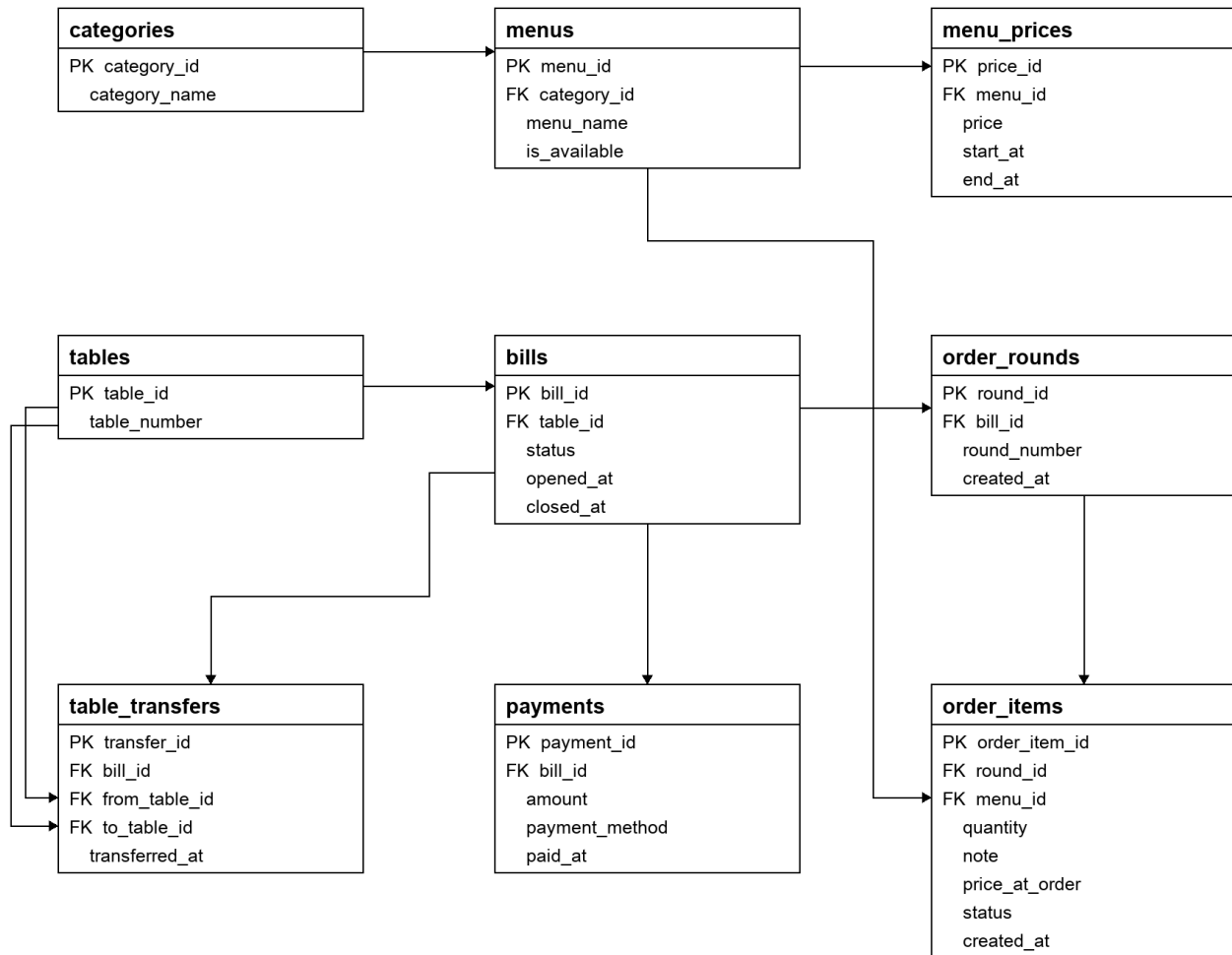
พิมลทิพ แซ่ยี่ 6721651637
กชกร เกิดมงคล 6721651106
กุลปรียา สุคำทัศน์ 6721651173
อมลวัฒน์ ลวกิตติไชยยันต์ 6721651912

React Native + Expo SDK 57 + SQLite
GitHub: <https://github.com/Joypmtsy/Restaurant>
อ้างอิงโค้ด ณ commit df87c77

1. แผนผังฐานข้อมูล

ระบบใช้ฐานข้อมูล restaurant-order-01418342.db ภายในเครื่องเดียว มี 9 ตาราง แผนผังระบุ PK เป็นคีย์หลัก และ FK เป็นคีย์นอก ลูกศรชี้จากตารางที่ถูกอ้างอิงไปยังตารางลูก โดยหัวไปหนึ่งแถวเชื่อมกับแถวลูกได้ตั้งแต่ศูนย์ถึงหลายแถว

Restaurant database



PK = Primary Key FK = Foreign Key

Arrow: parent table -> child table (one -> zero or many)

tables -> table_transfers: from_table_id and to_table_id

ความสัมพันธ์และเงื่อนไขสำคัญ

โต๊ะหนึ่งมีประวัติหลายบิล แต่มีบิลเปิดได้เพียงใบเดียวในเวลาเดียวกัน บิลหนึ่งมีหลายรอบ และรอบหนึ่งมีหลายรายการอาหาร เมนูหนึ่งมีประวัติหลายราคา แต่มีราคาปัจจุบันได้เพียงรายการเดียว การย้ายโต๊ะอ้างอิง tables สองครั้ง คือโต๊ะต้นทางและปลายทาง FK ใช้ ON DELETE RESTRICT เพื่อไม่ให้ลบข้อมูลหลักที่ยังถูกอ้างอิง ส่วน order_rounds มี UNIQUE(bill_id, round_number) ป้องกันเลขรอบซ้ำในบิลเดียวกัน การเปิดฐานข้อมูลใช้ SQLiteProvider จุดเดียวใน App.js และ Screen อ่านข้อมูลผ่าน src/db

2. เหตุผลของการแยกตาราง

ตาราง	เหตุผลที่แยก
tables / bills	โต๊ะเป็นข้อมูลใช้ซ้ำ แต่บิลเกิดใหม่ทุกครั้งที่ใช้งาน จึงเก็บประวัติบิลเก่าได้โดยไม่แก้ข้อมูลโต๊ะ
categories / menus	หนึ่งหมวดมีหลายเมนู ไม่ต้องเขียนชื่อหมวดซ้ำ และจัดหมวดจากรหัสเดียวกันได้
menus / menu_prices	ชื่อเมนูไม่เปลี่ยนทุกครั้งที่เปลี่ยนราคา จึงแยกประวัติราคาและช่วงเวลาของราคาออกมา
bills / order_rounds / order_items	แยกการใช้โต๊ะ รอบที่ส่ง และอาหารแต่ละรายการ เพื่อให้สั่งเพิ่มได้และแยกงานครัวตามรอบ
payments	ข้อมูลเงิน วิธีชำระ และเวลาชำระต่างจากรายละเอียดอาหาร จึงตรวจสอบการรับเงินแยกได้
table_transfers	เก็บเหตุการณ์ย้ายโต๊ะ โดยไม่ต้องคัดลอกบิลหรือรายการอาหารไปสร้างใหม่

ถ้ารวมทั้งหมดไว้ตารางเดียว ข้อมูลโต๊ะ บิล และเมนูจะซ้ำในทุกแถวอาหาร เมื่อแก้ไขอาจแก้ไขไม่ครบทุกแถว การแยกตารางแล้วเชื่อมด้วยรหัสช่วยลดข้อมูลซ้ำและความคุมความถูกต้องได้ง่ายกว่า

3. การเก็บราคา

ราคาเก็บเป็น INTEGER หน่วยบาทจำนวนเต็ม เช่น 150 บาทเก็บเป็น 150

กลุ่มเลือกหน่วยนี้เพื่อให้อ่านและคำนวณง่าย และไม่รองรับเศษสตางค์ ห้ามใช้ REAL เก็บราคา ส่วน menu_prices เก็บประวัติด้วย start_at และ end_at โดยแถวที่ end_at เป็น NULL คือราคาปัจจุบัน

เมื่อเปลี่ยนราคา setMenuPrice() ปิดช่วงราคาเดิมแล้วเพิ่มราคาใหม่ใน Transaction เดียวกัน ส่วน submitOrder() อ่านราคาปัจจุบันจาก SQLite และบันทึกใน order_items.price_at_order ยอดบิลคำนวณ SUM(quantity * price_at_order) จึงไม่อ่านราคาเมนูใหม่มาคิดกับอาหารเก่า

ตัวอย่าง: สั่งดื่มยำกุ้งราคา 150 บาท 2 รายการ จะเก็บ price_at_order = 150 และยอด 300 บาท แม้เปลี่ยนเมนูเป็น 160 บาท บิลเดิมยังเป็น 300 บาท เพราะราคาแต่ละรายการถูกบันทึกไว้แล้ว

4. การรองรับหลายรอบการสั่ง

order_rounds เชื่อม bills ด้วย bill_id และ order_items เชื่อมรอบด้วย round_id เมื่อสั่งเพิ่ม submitOrder() ใช้ COALESCE(MAX(round_number), 0) + 1 ของบิลนั้นสร้างเลขรอบใหม่ ทำให้รอบ 1 และ 2 อยู่ในบิลเดียวกัน แต่แสดงแยกชุดอาหารและเวลาสั่งได้

การรู้รอบช่วยให้ครัวแยกชุดที่เพิ่งส่งจากชุดเดิม ลูกค้านับได้ว่ารายการใดอยู่รอบไหน และรวมยอดทุกรอบด้วย bill_id ได้โดยไม่เปิดบิลใหม่ ตะกร้าก่อนส่งอยู่ใน CartContext และล้างเฉพาะเมื่อบันทึกสำเร็จ

5. SQL สามคำสั่งที่ซับซ้อนที่สุด

คำสั่งต่อไปนี้มาจากโค้ดจริง โดยจัดบรรทัดใหม่เพื่อให้อ่านง่าย แต่ไม่เปลี่ยนคอลัมน์ เงื่อนไข หรือการเรียงลำดับ

คำสั่งที่ 1: คิวครัว - src/db/kitchenDb.js / getKitchenQueue()

```
SELECT
  oi.order_item_id,
  oi.round_id,
  around.round_number,
  around.bill_id,
  b.table_id,
  t.table_number,
  oi.menu_id,
  m.menu_name,
  oi.quantity,
  oi.note,
  oi.status,
  oi.created_at
FROM order_items oi
INNER JOIN order_rounds around ON around.round_id = oi.round_id
INNER JOIN bills b ON b.bill_id = around.bill_id
INNER JOIN tables t ON t.table_id = b.table_id
INNER JOIN menus m ON m.menu_id = oi.menu_id
WHERE oi.status IN ( 'waiting', 'cooking' )
ORDER BY oi.created_at ASC, oi.order_item_id ASC
```

SELECT เลือกรหัสรายการ รอบ บิล โต๊ะ เมนู จำนวนหมายเหตุ สถานะ และเวลาสั่ง FROM เริ่มจากอาหารแต่ละรายการ INNER JOIN order_rounds หาเลขรอบและบิล JOIN bills หาโต๊ะของบิล JOIN tables หาเลขโต๊ะ และ JOIN menus หาชื่ออาหาร WHERE เลือกเฉพาะ waiting/cooking เพราะรายการเสร็จแล้วไม่ใช้งานค้าง ORDER BY เรียงเวลาสั่งก่อนหลัง และใช้รหัสรายการตัดสินเมื่อเวลาเท่ากัน

คำสั่งที่ 2: ยอดขายแยกหมวด - screenDb.js / reportForScreen()

```
SELECT
  c.category_id,
  c.category_name,
  SUM(oi.quantity) AS quantity,
  SUM(oi.quantity * oi.price_at_order) AS total
FROM bills b
JOIN order_rounds r ON r.bill_id = b.bill_id
JOIN order_items oi ON oi.round_id = r.round_id
JOIN menus m ON m.menu_id = oi.menu_id
JOIN categories c ON c.category_id = m.category_id
WHERE b.status = 'closed'
  AND julianday(b.closed_at) >= julianday(?)
  AND julianday(b.closed_at) < julianday(?)
GROUP BY c.category_id
ORDER BY total DESC
```

SELECT เลือกรหัส/ชื่อหมวด SUM(quantity) รวมจำนวนขาย และ SUM(quantity * price_at_order) รวมยอดจากราคาตอนสั่ง FROM bills เชื่อมรอบ อาหาร เมนู และหมวดด้วย JOIN ... ON WHERE เลือกบิล closed จากนั้นกำหนดเวลาเริ่มแบบรวมขอบ และเวลาสิ้นสุดแบบไม่รวมขอบ GROUP BY แยกผลตามหมวด ORDER BY เรียงหมวดที่มียอดขายสูงก่อน จึงใช้ดูว่าหมวดใดสร้างยอดมากที่สุดได้

คำสั่งที่ 3: ประวัติบิลและยอดรวม - screenDb.js / reportForScreen()

```
SELECT
  b.bill_id,
  t.table_number,
  b.opened_at,
  b.closed_at,
  COALESCE(SUM(oi.quantity * oi.price_at_order), 0) AS total
FROM bills b
JOIN tables t ON t.table_id = b.table_id
LEFT JOIN order_rounds r ON r.bill_id = b.bill_id
LEFT JOIN order_items oi ON oi.round_id = r.round_id
WHERE b.status = 'closed'
  AND julianday(b.closed_at) >= julianday(?)
  AND julianday(b.closed_at) < julianday(?)
GROUP BY b.bill_id
ORDER BY b.closed_at DESC
```

SELECT เลือกข้อมูลบิล/โต๊ะ และรวมราคาตอนสั่ง COALESCE เปลี่ยนผลรวม NULL เป็น 0 JOIN tables หาเลขโต๊ะ LEFT JOIN รอบและอาหารยังคงบิลที่ไม่มีอาหารไว้ WHERE เลือกเฉพาะบิลปิดในช่วงวันที่ GROUP BY รวมเป็นหนึ่งแถวต่อบิล ORDER BY แสดงบิลที่ปิดล่าสุดก่อน จึงแสดงประวัติได้โดยไม่ทำยอดบิลซ้ำต่อรายการอาหาร

6. ดัชนีและผล EXPLAIN QUERY PLAN

ชื่อ Index / คอลัมน์	เหตุผล
ux_menu_prices_current / menu_id	UNIQUE เฉพาะ end_at IS NULL: หาราคาปัจจุบันและป้องกันราคาปัจจุบันซ้ำ
ux_bills_one_open_per_table / table_id	UNIQUE เฉพาะ status = open: หาบิลเปิดและป้องกันบิลเปิดซ้ำต่อโต๊ะ
idx_order_items_status / status	กรองรายการอาหารที่ครัวต้องทำ
idx_order_items_created / created_at	รองรับการค้นหารายการตามเวลา
idx_order_rounds_created / created_at	รองรับการค้นหารอบตามเวลา
idx_bills_closed_at / closed_at	รองรับช่วงเวลาปิดบิลเมื่อเปรียบเทียบคอลัมน์โดยตรง

ทดสอบด้วย EXPLAIN QUERY PLAN ตามด้วย SQL จริงของแต่ละฟังก์ชัน ใน SQLite 3.53.1 ใช้ schema.sql และข้อมูลตั้งต้น ไม่มีข้อมูลขายและไม่ได้รัน ANALYZE ผลเป็นหลักฐานแผนการอ่าน ไม่ใช่ผลจับเวลาบน Expo Go และอาจต่างตามเวอร์ชันหรือจำนวนข้อมูล

คิวครัว getKitchenQueue(): ใช้ดัชนีสถานะจริง

```
SEARCH oi USING INDEX idx_order_items_status (status=?)
SEARCH around USING INTEGER PRIMARY KEY (rowid=?)
SEARCH b USING INTEGER PRIMARY KEY (rowid=?)
SEARCH t USING INTEGER PRIMARY KEY (rowid=?)
SEARCH m USING INTEGER PRIMARY KEY (rowid=?)
USE TEMP B-TREE FOR ORDER BY
```

getTables() และ getMenu(): ใช้ดัชนีบิลเปิดและราคาปัจจุบัน

```
SEARCH b USING COVERING INDEX ux_bills_one_open_per_table (table_id=?) LEFT-JOIN
SEARCH mp USING INDEX ux_menu_prices_current (menu_id=?) LEFT-JOIN
```

SEARCH คือค้นหาเฉพาะส่วนตามเงื่อนไข ส่วน TEMP B-TREE คือพื้นที่ชั่วคราวสำหรับเรียงข้อมูล คำสั่งรายงานบิลพบ SCAN b จึงยังไม่ใช่ idx_bills_closed_at กรองวันที่ เพราะเงื่อนไขใช้ julianday(closed_at) ไม่ใช่คอลัมน์ตรง ๆ ไม่ควรสรุปว่าทุก Index ถูกใช้ในทุกหน้า

7. การใช้ Transaction

transaction.js ใช้ db.withTransactionAsync() รวมงานเขียน และ writeQueue ซึ่งเป็นตัวแปร Promise จัดให้การเขียนผ่านตัวกลางทำทีละงาน เมื่อ task ทำครบจะบันทึก แต่ถ้าโยน Error จะย้อนกลับการเปลี่ยนแปลงของงานนั้น ไม่กระทบงานที่สำเร็จก่อนหน้านี้

ฟังก์ชัน	เหตุผล / ปัญหาถ้าไม่ใช้
submitOrder	รอบและอาหารต้องครบชุด ถ้าเมนูหนึ่งผิดพลาดอาจเหลือรอบว่างหรืออาหารบางรายการ
setMenuPrice	ปิดราคาเดิมกับเพิ่มราคาใหม่ต้องสำเร็จด้วยกัน ไม่เช่นนั้นเมนูอาจไม่มีราคาปัจจุบัน
moveTable	เปลี่ยนโต๊ะของบิลกับบันทึกประวัติต้องตรงกัน ไม่เช่นนั้นย้ายแล้วแต่ไม่มีประวัติ
settleBill	ข้อมูลชำระกับการปิดบิลต้องตรงกัน ไม่เช่นนั้นมีการชำระแล้วแต่บิลยังเปิด
clearTransactions	ลบข้อมูลขายทุกตารางเป็นชุดเดียว ป้องกันลบสำเร็จเพียงบางตาราง
initializeRestaurant / openTable / setMenuAvailable / advanceKitchenItem	เตรียมข้อมูล เปิดบิล และเปลี่ยนข้อมูลผ่านคิวเดียวกัน โดยยังตรวจเงื่อนไขและผลการอัปเดต

ตัวอย่างการเปลี่ยนราคา: ถ้าปิดราคา 150 บาทแล้วเพิ่มราคา 160 บาทไม่สำเร็จ Transaction จะคืนราคา 150 บาทให้ยังเป็นราคาปัจจุบัน จึงไม่มีช่วงข้อมูลค้างที่ใช้ขายไม่ได้ ส่วน UNIQUE, FK และการตรวจค่าทำหน้าที่ควบคุมข้อมูลร่วมกับ Transaction

8. ปัญหาที่พบและวิธีแก้

ปัญหาที่ 1: iOS สร้าง bundle ไม่สำเร็จ

อาการ: ขึ้น Invalid call: process.env.EXPO_ROUTER_APP_ROOT และ require.context ต้องการข้อความระบุโฟลเดอร์ **สาเหตุ:** ดอนใช้ Expo Router ค่าโฟลเดอร์หน้าจอไม่ได้ถูกแทนเป็น string ในขึ้นแปลงโค้ด จึงโหลด routes ไม่ได้ หลักฐาน error ไม่ระบุว่าตัวตั้งค่าใดผิดเพียงตัวเดียว **วิธีแก้:** เมื่อกลุ่มเลือกใช้ App.js จึงเปลี่ยน entry เป็น index.js ที่ registerRootComponent(App) และใช้ NavigationContext ให้การเปลี่ยนหน้าสอดคล้องกัน จากนั้นตรวจ bundle iOS/Android ผ่าน

ปัญหาที่ 2: กดปุ่มแล้วไม่เปลี่ยนหน้า

อาการ: หน้าแรกเปิดได้ แต่กดเลือกบทบาแล้วหน้าจอดัง **สาเหตุ:** state การนำทางอยู่เหนือ SQLiteProvider ที่ memo โดยการเปลี่ยน children ไม่ทำให้ Provider แสดงเนื้อหาใหม่ตามที่คาด **วิธีแก้:** ย้าย state และการนำทางไว้ใน RestaurantScreens ภายใน Provider แล้วตรวจการเปิดหน้าลูกค้า คร้ว และผู้ดูแลบน Simulator

ปัญหาที่ 3: push งานขึ้น GitHub ถูกปฏิเสธ

อาการ: git push ขึ้น rejected (fetch first) **สาเหตุ:** main บน GitHub มี commit ของเพื่อนที่เครื่องยังไม่มี จึง push แบบ fast-forward ไม่ได้ **วิธีแก้:** fetch และรวมประวัติ แก้ conflict ของ Screen/Components โดยตรวจพฤติกรรมจริง แล้วตรวจ lint และ bundle ก่อน push เพื่อเก็บประวัติสมาชิกครบ

9. การแบ่งงานและสัดส่วน

แบ่งตามหน้าที่และไฟล์ที่พบใน Git ไม่ได้อ้างว่าหนึ่งไฟล์มีผู้เขียนเพียงคนเดียว เพราะช่วงรวมระบบมีการแก้ไฟล์ร่วมกัน สัดส่วนด้านล่างเป็นร่างประมาณการ 40/20/20/20 ที่ต้องให้สมาชิกยืนยันก่อนส่ง ไม่ใช่สัดส่วนที่คำนวณจากจำนวน commit

สมาชิก / รหัส	งานและหลักฐาน Git	สัดส่วนร่าง
พิมลทิพ แซ่ยี่ 6721651637	ฐานข้อมูล seed ส่วนกลาง styles รวมระบบและแก้การนำทาง หลักฐาน: เพิ่มไฟล์ส่วนกลาง, ปรับหน้าจอและการนำทาง	40%
อมลวัฒน์ ลวกิตติไชยยันต์ 6721651912	CTableScreen, CMenuScreen, CartScreen และ Components หลักฐาน: commit เดิม 0ab1fbd, 31b28df, 3132795	20%
กุลปรียา สุคำทัศน์ 6721651173	KQueueScreen, KDetailScreen, COrderScreen, CBillScreen หลักฐาน: 2f1a067, bb718d4, dc9bb9a	20%
กชกร เกิดมงคล 6721651106	ATableScreen, AMenuScreen, ABillScreen และภาพรวม/รายงาน/ล้างข้อมูล หลักฐาน: 1715d0b, 74953e0, 72de65b	20%

ณ commit df87c77 git shortlog รวม merge พบ pim.moler 22 commit, amwttt 12, kochkorn 7 และ focusonmindmine 5 จำนวน commit บอกรายการที่ทำงาน ไม่ใช่เวลาหรือปริมาณงานจริง จึงใช้ประกอบกับไฟล์และหน้าที่ การแก้ข้อความ commit ช่วงท้ายคงผู้เขียนวันที่ และเนื้อหาเดิมไว้ แต่ทำให้รหัส commit บางรายการเปลี่ยน

ฐานข้อมูลเป็นส่วนกลางที่ทุกหน้าต้องใช้ สมาชิกแต่ละคนต้องเข้าใจ SQL ที่หน้าของตนเรียกใช้ และตรวจหลักฐานการเขียน SQL อย่างน้อยคนละ 3 คำสั่งตามโจทย์ การแบ่งส่วนอธิบายในคลิปไม่ใช่หลักฐานว่าเป็นผู้เขียนคำสั่งนั้นทั้งหมด

10. สิ่งที่จะทำต่อถ้ามีเวลา

1) ยกเลิกอาหารที่ยังไม่เริ่มทำ

พัฒนาตามโจทย์ระดับ ข ข้อ ข3 ให้ยกเลิกได้เฉพาะอาหารที่สถานะยังเป็นรอทำ และบันทึกเวลายกเลิกไว้ เพื่อให้ลูกค้าแก้รายการที่สั่งผิดได้โดยไม่กระทบอาหารที่ครัวเริ่มทำแล้ว พร้อมปรับคิวครัวและยอดบิลให้ตรงกับรายการที่ยังใช้งานอยู่

2) เพิ่มเมนูใหม่ผ่านแอป

ต่อยอดโจทย์ระดับ ข ข้อ ข5 โดยเพิ่มหน้ากรอกชื่อเมนู เลือกหมวด และกำหนดราคาเริ่มต้น ปัจจุบันระบบแก้ราคาและเปิดหรือปิดขายเมนูเดิมได้ แต่ยังเพิ่มเมนูใหม่ผ่านหน้าจอก็ไม่ได้ การเพิ่มส่วนนี้ช่วยให้ผู้ดูแลจัดการเมนูได้โดยไม่ต้องแก้ข้อมูลตั้งต้นในโค้ด

3) ออกใบเสร็จเป็นไฟล์

พัฒนาตามโจทย์ระดับ ค ให้สร้างใบเสร็จเป็นไฟล์หลังปิดบิล โดยแสดงเลขบิล โต๊ะ รายการอาหาร จำนวน ราคาตอนสั่ง ยอดรวม และข้อมูลการชำระ เพื่อให้เก็บหลักฐานหรือส่งให้ลูกค้าได้ โดยอ่านข้อมูลจากบิลเดิมเพื่อให้ยอดตรงกับฐานข้อมูล